

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Laboratory Report

Operating Systems

Lab 1: The OS as a Resource Manager
Lab 2: Meet the OS You Will Build

Student: Cosneanu Corina
Group: FAF-241

Verified by:
Reitman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 Lab 1: The OS as a Resource Manager **2**
1.1 Part 1: File and Directory Management 2
1.2 Part 2: Process Scheduling and Management 2
1.3 Part 3: Memory Distribution 3
1.4 Part 4: Device Abstraction and Storage 3

2 Lab 2: Meet the OS You Will Build (xv6) **4**
2.1 Part 1 & 2: Compiling and Exploring the xv6 Environment 4
2.2 Part 3: Source Code Inspection 4
2.3 Part 4: Developing the `sleep` Utility 4

3 Conclusion **5**

1 Lab 1: The OS as a Resource Manager

1.1 Part 1: File and Directory Management

Task Execution:

```
$ mkdir -p ~/os-lab1 && cd ~/os-lab1
$ mkdir demo && cd demo
$ echo "hello operating systems" > note.txt
$ cp note.txt copy.txt
$ mv copy.txt renamed.txt
$ chmod 600 note.txt
$ ls -l
```

Analytical Observations:

- **Ownership Details:** Based on the terminal output, the files generated during this exercise are strictly owned by the `corina` user and assigned to the `corina` primary group.
- **Permission String (`-rw----`):** The `chmod 600` command restricts access drastically. The initial hyphen indicates a standard text file (not a directory). The following `rw-` segment grants read and write capabilities exclusively to the file owner. The subsequent six hyphens confirm that all group members and external users are denied any access rights.
- **Core Root Directories:** Inspecting the root directory reveals key architectural folders. For instance, `/etc` functions as the central repository for global system configurations. Meanwhile, `/dev` acts as a virtual filesystem where physical and virtual hardware devices are exposed as accessible files.

1.2 Part 2: Process Scheduling and Management

Task Execution:

```
$ sleep 300 &
$ PID=$!
$ cat /proc/$PID/status | head -10
$ kill $PID
```

Analytical Observations:

- **Process States:** When observing the `sleep` command within the `/proc` directory, its state is marked as `S (sleeping)`. This indicates an interruptible sleep phase, meaning the process consumes negligible CPU cycles while waiting for a specific timer to elapse.
- **The First Process (PID 1):** On modern Linux distributions, PID 1 is typically assigned to `systemd` (or `init`). It is the very first user-space process initialized by the kernel during boot, acting as the ultimate ancestor for all subsequent processes.

1.3 Part 3: Memory Distribution

Task Execution:

```
$ free -h
$ cat /proc/meminfo | head -5
$ grep VmRSS /proc/$PID/status
```

Analytical Observations:

- **System Memory Overview:** The `free -h` output illustrates the distribution of the system's RAM. The OS actively separates memory into used space, completely unallocated (free) space, and cached memory (which can be rapidly reallocated if applications demand more resources).
- **Understanding Swap:** Swap space serves as an emergency overflow residing on the physical hard drive. Whenever the primary RAM reaches capacity, the OS moves dormant memory pages into this swap area to prevent system crashes, though accessing it is significantly slower.
- **Resident Memory of `sleep`:** Investigating the `VmRSS` (Virtual Memory Resident Set Size) metric shows that the `sleep` command occupies around 1.5 MB of physical memory. Even for a command that performs no active computations, this memory footprint is required to map the standard C library and essential binary metadata.

1.4 Part 4: Device Abstraction and Storage

Task Execution:

```
$ lsblk
$ df -h
$ ls -l /dev | head -10
```

Analytical Observations:

- **Root Mount Point:** The `df -h` and `lsblk` utilities confirm that the primary root file system (`/`) is mounted on a specific block device (e.g., `/dev/sdd` or a similar virtual disk allocation depending on the WSL/VM environment).
- **The UNIX Philosophy:** The phrase "Everything is a file" describes the design paradigm where peripherals, network sockets, and storage drives are abstracted into standard file interfaces. As observed in the `/dev` folder, hardware endpoints are listed identically to standard text files, allowing developers to interact with them using standard `read()` and `write()` system calls.

2 Lab 2: Meet the OS You Will Build (xv6)

2.1 Part 1 & 2: Compiling and Exploring the xv6 Environment

Task Execution:

```
$ make qemu
# Inside xv6:
$ ls
$ echo "Testing xv6"
$ cat README
```

Analytical Observations:

- **Available Utilities:** Unlike a full Linux distribution, the educational xv6 OS ships with a highly reduced toolset. The executable programs available natively include `cat`, `echo`, `grep`, `ls`, and `wc`.
- **Pipeline Requirements:** Executing a command like `ls | wc` relies on two fundamental OS features: the ability to generate concurrent execution threads (process creation via `fork`) and the ability to route data between them securely (inter-process communication via kernel `pipes`).
- **Shell Interface:** The default `sh` program in xv6 feels remarkably similar to a standard Linux terminal because it implements the same foundational POSIX standards. However, it lacks modern quality-of-life enhancements such as command history and tab-completion.

2.2 Part 3: Source Code Inspection

Task Execution:

```
$ sed -n "1,30p" user/cat.c
$ grep -n "sys_read" kernel/sysfile.c
```

Analytical Observations:

- **System Calls in `cat.c`:** The source code for the `cat` utility depends heavily on three primary system calls: `read()` to capture bytes from the specified file, `write()` to push those bytes to the terminal screen, and `exit()` to safely terminate the program once the file is fully processed.
- **Kernel vs. User Space:** The xv6 architecture is cleanly divided. The `kernel/` directory contains the core privileged instructions that interact directly with the CPU and memory management unit. Conversely, the `user/` directory stores standard applications that run in an isolated, unprivileged mode. These applications must use software interrupts to ask the kernel to perform operations on their behalf.

2.3 Part 4: Developing the `sleep` Utility

To expand the capabilities of xv6, a custom `sleep` program was implemented in the user space.

Implementation of `user/sleep.c`:

```c
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[]) {
    // Ensure the user provided exactly one argument
    if (argc != 2) {
        fprintf(2, "Error: usage is sleep <ticks>\n");
        exit(1);
    }

    // Convert the string argument to an integer and call the system
    sleep
    int ticks = atoi(argv[1]);
    sleep(ticks);

    exit(0);
}
```

Integration: To compile the new utility, the `Makefile` was modified by appending `$U/_sleep\` to the `UPROGS` directive. Running `make qemu` successfully compiled the binary, which functioned exactly as expected when tested in the xv6 shell.

3 Conclusion

These introductory laboratories successfully established a foundational understanding of operating system mechanics. By analyzing a live Linux environment in Lab 1, the abstract concepts of process scheduling, virtual memory paging, and hardware abstraction were visualized through practical terminal commands. Subsequently, compiling and modifying the xv6 kernel in Lab 2 demystified the separation between user-space applications and privileged kernel operations. Implementing a custom utility reinforced the concept of system calls as the critical bridge between application logic and hardware resources.